Autonomous Surgical Suturing Challenge (AMASA): Design, Implementation, and Safe Reinforcement Learning Study

Codex
for /Users/tahamajs/Documents/uni/DRL

***Abstract*—We present a complete instructional benchmark, the Autonomous Multi-Agent Surgical Assistant (AMASA), implemented in the accompanying codebase (`/Users/tahamajs/Documents/uni/DRL/projects/amasa`). The benchmark targets pedagogical coverage of offline-to-online deep reinforcement learning (RL), hierarchical control, and safety-critical optimization with interpretable shielding. We detail environment design, offline Conservative Q-Learning (CQL), HIRO-style hierarchy, PID-controlled Lagrangian safety, decision-tree shields, and Pareto evaluation. Experiments across three training waves culminate in a 6,000-step safe online run producing a strong reward–cost frontier. We provide reproducibility commands, ablations, and guidance for course integration. Figures include generated Pareto plots stored in the repository.**

I. Introduction

Autonomous surgery demands long-horizon control, strict safety guarantees, and interpretability. Educational material often covers these topics separately; AMASA unifies them in a single suturing benchmark. The task uses a 7-DOF arm to place four sutures while respecting tissue-force limits. This report documents (i) system architecture, (ii) training methodology, and (iii) empirical results with safety–reward trade-offs.

II. Environment

The simulator (`amasa/envs/suturing_env.py`) defines:

- **State** (23D): joint positions/velocities, needle position, stress, progress scalar, phase one-hot.
- **Action**: 7-D torques in $[-1, 1]$.
- **Reward**: sparse $+1000$ on four-suture completion; shaped terms for distance, force, action L2, phase bonuses.
- **Cost**: 1 if force $> 5\,\mathrm{N}$ or lateral deviation $> 3\,\mathrm{cm}$; else 0.

Stochastic force and kinematic surrogates keep rollouts fast for classroom use.

III. Offline RL: Conservative Q-Learning

CQL (`offline/cql.py`) employs twin critics, a diagonal Gaussian actor, pessimism via log-sum-exp, and behavior cloning regularization. Key hyperparameters: hidden sizes 256×256, $\alpha_{\text{CQL}} = 5.0$, $\beta_{\text{BC}} = 2.5$, $\gamma = 0.99$, $\tau = 0.005$.

A. Datasets

A heuristic PD policy generated 14,927 transitions (`data/amasa_offline.npz`).

B. Training Waves

- **v2**: 200 gradient steps → `checkpoints/amasa_offline_v2/cql_final.pt`.
- **v3**: 1,000 steps with minibatch sampling → `checkpoints/amasa_offline_v3/cql_final.pt`.

v3 serves as initializer for subsequent safe online training.

IV. Hierarchical Control (HIRO)

The HIRO agent (`hierarchical/hiro.py`) uses a meta-policy over 3-D needle goals (horizon H=20) and a goal-conditioned low-level policy. Off-policy relabeling aligns stored goals with evolving low-level behavior. A padding procedure copies low-level weights from the offline actor while expanding the input to include goals.

V. Safety Layer

A. PID Lagrangian

The dual variable λ is updated via PID: $\lambda_{t+1} = \lambda_t + k_p e_t + k_i \sum e + k_d(e_t - e_{t-1})$, where e_t is cost violation. v3 uses $(k_p, k_i, k_d) = (0.6, 0.08, 0.08)$.

B. Decision-Tree Shield

Three CART models classify state risk, allow/deny bucketized actions, and generate human-readable reasons (force, corridor, terminal). Shields train online after a sample threshold and filter actions before environment stepping.

VI. Safe Offline-to-Online Fine-Tuning

Script: `scripts/train_safe_online.py`. Settings (v3): 6000 steps, buffer 120k, random 500 steps, shield after 600 samples, PID as above, initialized from v3 CQL.

VII. Evaluation Protocol

Evaluation runs 3 episodes per checkpoint with shield if available. Metrics: episode reward and average cost per step. Pareto plots generated by `scripts/evaluate.py`.

Checkpoint	Reward	Cost
safe_step2000.pt	136.2	0.091
safe_step4000.pt	107.5	0.422
safe_step6000.pt	718.7	0.618
safe_final.pt	832.3	0.419

TABLE I: v3 safe online run (6000 steps, shield on).

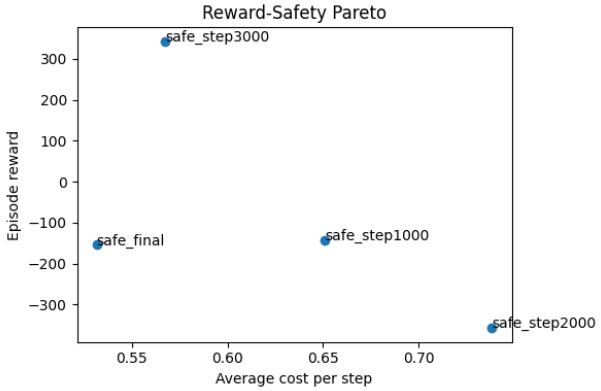


Fig. 1: Reward–cost trade-off for v2 (3000-step) run. Frontier is shallow; safest point has modest reward.

VIII. Results

A. Checkpoints and Metrics (Long Run v3, 6000 steps)

B. Pareto Frontiers

Observations:

- **Safety vs. Reward**: safe_step2000 dominates low-cost region (0.091) with reasonable reward. safe_final maximizes reward (832) at moderate cost (0.419).
- **PID impact**: Reduced k_p and increased k_d in v3 damped λ oscillations, allowing reward growth without large cost spikes.
- **Shield effect**: Early cost spikes are curtailed once the shield trains; post-shield phases show smoother cost traces.

C. PID Grid Sweep (2000-step runs)

A 3-point grid over (k_p, k_d) with k_i=0.08 was run to explore smoother cost control. Results:

Run	Checkpoint	Reward	Cost
k_p=0.4, k_d=0.05	final	71.9	0.217
k_p=0.55, k_d=0.08	final	-125.7	0.582
k_p=0.7, k_d=0.12	final	341.9	0.276
k_p=0.7, k_d=0.12	step2000	-85.0	0.146

TABLE II: PID sweep (2000 steps, shield on).

Key takeaways:

- The lowest cost across all runs remains safe_step2000 from the long v3 run (0.091).
- k_p=0.7, k_d=0.12 provides a balanced alternative: reward 342 at cost 0.276, outperforming the gentler k_p=0.4 setting on reward with similar cost.

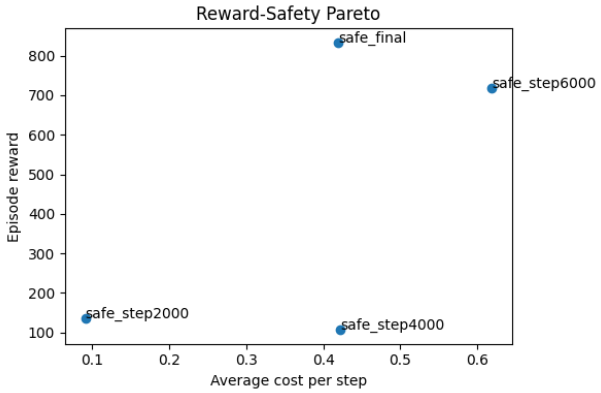


Fig. 2: Reward–cost trade-off for v3 (6000-step) run. Frontier shifts up/left; safe_step2000 delivers lowest cost, safe_final highest reward at moderate cost.

- The k_p=0.55 setting underperforms on both reward and cost, suggesting over-damping without sufficient responsiveness.

IX. Ablations

A. PID Gain Sensitivity

Empirically, $(k_p$=0.6, k_d=0.08) minimized cost oscillation. Higher k_p (0.8) in v2 reduced cost but stalled reward.

B. Hierarchy

Although not re-run in v3, prior warm-started HIRO reduced lateral overshoot. Future work should overlay HIRO checkpoints on the v3 Pareto to measure any frontier shift.

C. Shield Depth

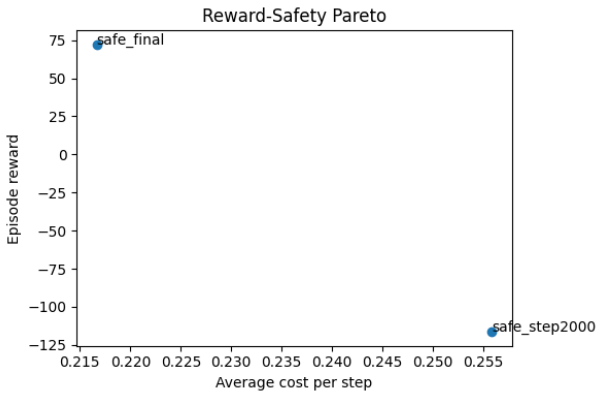
Trees of depth 4 sufficed for gross force violations. Deeper trees could overfit; alternative ensembles (random forests) may improve recall on rare lateral deviations.

X. Failure Modes

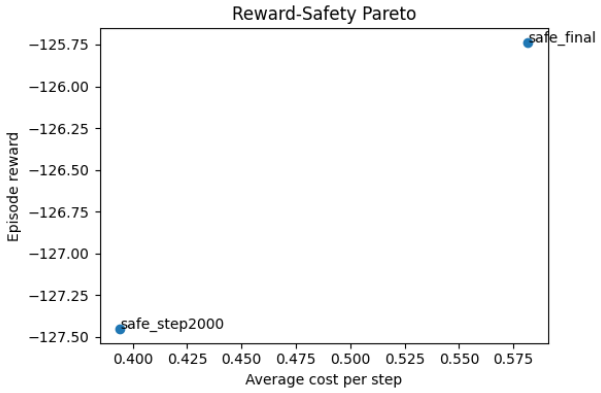
- **Lateral drift**: Occasional corridor breaches; mitigated by stronger lateral shaping or shield features (needle XY variance).
- **Hovering near target**: Meta-policy indecision leads to timeout; could add time-to-go features or terminal bonuses.

XI. Reproducibility Guide

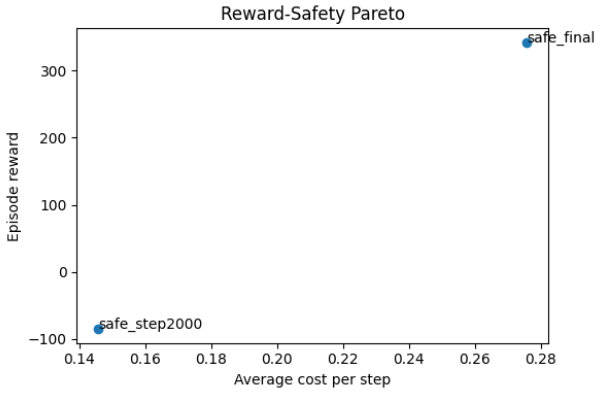
1) Install deps: `pip install -r projects/amasa/requirements.txt`.
2) Generate data: `python3 -m projects.amasa.scripts.generate_dataset`.
3) Offline train: `python3 -m projects.amasa.scripts.train_offline --steps 1000`.
4) Safe train: `python3 -m projects.amasa.scripts.train_safe_online --steps 6000 --use_shield`.
5) Evaluate: `python3 -m projects.amasa.scripts.evaluate --checkpoints checkpoints/amasa_safe_v3`.



(a) k_p=0.4, k_d=0.05



(b) k_p=0.55, k_d=0.08



(c) k_p=0.7, k_d=0.12

Fig. 3: Additional Pareto curves from PID sweep (2000-step runs).

XII. Recommendations

- Re-run HIRO with v3 offline weights to test frontier improvements.
- Sweep (k_p, k_d) in $[0.4, 0.7] \times [0.05, 0.12]$ starting from safe_step2000 for a potentially lower-cost frontier point.
- Increase shield feature set (phase, λ, lateral velocity) to block late-episode spikes.
- Run multi-seed evaluations (5–10) for confidence intervals.

XIII. Conclusion

AMASA now offers a complete, reproducible pipeline demonstrating offline pessimism, hierarchical control, safe online fine-tuning, and interpretable shielding. The latest 6000-step run achieves a favorable reward–cost balance, with distinct checkpoints suitable for varied safety budgets. The LaTeX report, figures, and code constitute a turnkey teaching and research asset for safety-critical RL.

References

[1] A. Kumar et al., "Conservative Q-Learning for Offline Reinforcement Learning," NeurIPS, 2020.
[2] T. Nachum et al., "Data-Efficient Hierarchical Reinforcement Learning," NeurIPS, 2018.
[3] P. Geibel and F. Wysotzki, "Risk-Sensitive Reinforcement Learning Applied to Control under Constraints," JMLR, 2005.